

Multiple Inheritance in Python

1. How the super() Function Handles Multiple Inheritance

The `super()` function in Python provides a way to call methods from parent classes in a manner that works seamlessly with multiple inheritance. It relies on Python's Method Resolution Order (MRO) to determine which method to call. Let's explore how it works with an example:

```
class A:
    def method(self):
        print("A.method called")

class B(A):
    def method(self):
        print("B.method called")
        super().method() # This calls A.method

class C(A):
    def method(self):
        print("C.method called")
        super().method() # This calls A.method

class D(B, C):
    def method(self):
        print("D.method called")
        super().method() # This calls B.method, not C.method

x=D()
x.method()
```

In this example, the MRO for class D is [D, B, C, A, object]. When **`super().method()`** is called from class D, it calls the next method in the MRO, which is B.method. Then inside B `super().method()` calls the next method in the MRO which is C.method and so on.

output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ahmed> & C:/Users/ahmed/anaconda3/python.exe c:/Users/ahmed/Untitled-1.py
D.method called
B.method called
C.method called
A.method called
PS C:\Users\ahmed> 
```

```
print(D.__mro__)  
# Output: (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,  
<class '__main__.A'>, <class 'object'>)
```

2. Method Conflict Resolution in Multiple Inheritance

When a class inherits from multiple parent classes that have methods with the same name, Python resolves conflicts using the Method Resolution Order (MRO).

```
class Mammal:  
    def eat(self):  
        print("Mammal eating method: Consuming food for energy")  
  
class Human:  
    def eat(self):  
        print("Human eating method: Using utensils to eat cooked food")  
  
class Employee(Human, Mammal):  
    def eat(self):  
        print("Employee eating method: Eating lunch in the office")  
        super().eat() # This will call Human.eat()  
  
emp = Employee()  
emp.eat()
```

output

```
19  
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\ahmed> & C:/Users/ahmed/anaconda3/python.exe c:/Users/ahmed/Untitled-1.py  
Employee eating method: Eating lunch in the office  
Human eating method: Using utensils to eat cooked food  
PS C:\Users\ahmed> █
```

When `super().eat()` is called from the `Employee` class, it only calls `Human.eat()` and stops there. This happens because `Human.eat()` doesn't have a call to `super().eat()` inside it, so the method resolution chain stops at that point.

Sources :

<https://docs.python.org/3/library/functions.html#super>

<https://realpython.com/python-super/>